

Cost-Aware Conditional Diffusion Compression of 4K-5K Drone Imagery for Scalable Disaster-Response HPC Workflows

Jooho Kim*, Anish Shakya†, Yifan Yang†, and Jacob Kelly†

*Institute for a Disaster Resilient Texas, Texas A&M University, College Station, TX, USA

†Texas A&M University, College Station, TX, USA

Corresponding author: jooho.kim@tamu.edu

Abstract—High-resolution drone imagery is increasingly important for disaster response because it captures local scene evidence that complements satellite and street-view observations. The same resolution that makes these images useful also makes them expensive to store, move, and reconstruct in high-performance computing (HPC) workflows. This paper studies conditional diffusion compression (CDC) as a cost-aware compression and reconstruction strategy for 4K-5K drone imagery on GPU HPC systems. We do not propose a new neural compression architecture. Instead, we ask which operational controls make an existing CDC workflow practical for disaster-imagery workloads. We adapt an x-parameterized CDC implementation to NCSA DeltaAI, evaluate full-resolution drone images cropped to 5440×3648 pixels, and measure runtime, peak GPU memory, estimated bits per pixel (BPP), compression ratio, PSNR, SSIM, pixel-error statistics, and tile-boundary artifacts. Native full-image processing is feasible but expensive: at 65 denoising steps, the baseline uses about 52 GB of GPU memory and takes about 144 seconds per image. A selected 256×256 tiling configuration reduces runtime from 144.35 to 79.84 seconds per image and peak memory from 52.01 to 1.66 GB on 50 full-resolution images, while keeping scalar quality close to the no-tiling and 512×512 tiling references. Resolution reduction provides larger raw speedups, but it removes visual detail and degrades SSIM substantially at 2K and 1K settings. Checkpoint choice changes the storage-quality trade-off, while larger controlled-resolution batches and local storage staging do not improve throughput in the measured runs. The results show that CDC deployment for disaster imagery must be reported as a joint systems and image-quality problem: bitrate alone is not enough.

Index Terms—conditional diffusion compression, drone imagery, disaster response, high-performance computing, neural compression, tiling, GPU memory

I. Introduction

Drone imagery has become a practical source of post-disaster visual evidence. It can capture road blockages, roof damage, flood debris, and parcel-scale changes at a level of detail that coarser remote sensing products often cannot provide. This role is adjacent to recent disaster-imagery work that uses generative cross-view synthesis to reason from satellite to street-level views [1] and bi-temporal street-view imagery to support hyperlocal damage assessment [2]. Together, these lines of work point to a broader need: disaster-response systems need

visual data that are detailed enough for inspection, but inexpensive enough to store, move, and process repeatedly.

High-resolution drone images create a systems problem before they become a modeling problem. The images in our workflow are cropped to 5440×3648 pixels. A single 8-bit RGB frame at this resolution contains about 59.5 MB of uncompressed pixel data. A field collection with thousands of frames can therefore become a multi-hundred-gigabyte or terabyte-scale archive. In an HPC setting, the cost appears in several places: long-term storage, transfer from project filesystems to compute nodes, GPU memory pressure, queue time, and repeated decoding when analysts rerun quality checks or downstream models.

Learned image compression offers one path through this bottleneck. Autoencoder-based neural codecs and hyperprior models have shown strong rate-distortion performance [3]. Diffusion models provide a different reconstruction mechanism: they can generate visually plausible details through iterative denoising [4], and latent diffusion models reduce the cost of high-resolution synthesis by operating in a compact latent space [5]. Conditional diffusion compression (CDC) uses a diffusion decoder as part of a lossy compression system [6]. That design is attractive for visual quality, but it introduces a practical deployment issue: diffusion decoding is iterative, and full-resolution reconstruction can be slow and memory-heavy.

This paper studies CDC as an HPC workflow rather than as a new compression model. We ask: Which runtime settings make CDC usable for large drone-imagery workloads while preserving enough visual detail for disaster-response inspection? We focus on operational choices that a research team can control without retraining the model: denoising steps, precision, resolution, tile size, tile batch size, checkpoint choice, job parallelism, and storage placement.

Our contributions are:

- We adapt a CDC compression and reconstruction workflow to NCSA DeltaAI and define a reproducible experiment suite for high-resolution drone imagery.
- We report the cost-quality behavior of native full-resolution processing, denoising-step reduction, resolution reduction, tiling, batch-size changes, check-

point selection, storage placement, and parallel shards.

- We identify 256×256 tiling as the best current runtime-memory operating point in the selected validation run, reducing peak GPU memory by $31.3\times$ and runtime by 44.7% relative to no tiling.
- We show why visual QA is necessary for tiled CDC: scalar metrics are useful, but heatmaps, comparison panels, and seam checks are needed to detect spatial artifacts that would matter in disaster imagery.

II. Related Work

A. Disaster Imagery and Visual Evidence

Disaster-response imagery spans satellite, aerial, drone, street-view, and field observations. Each modality carries a different trade-off between coverage, resolution, timeliness, and interpretability. Satellite imagery provides broad coverage, but it may miss street-level details. Street-view imagery gives a ground perspective, but it may not be available immediately after an event. Yang et al. [1] study post-disaster satellite-to-street view synthesis and emphasize the realism-fidelity tension in generative disaster imagery. Yang et al. [2] use pre- and post-disaster street-view imagery with pretrained vision models for hyperlocal damage assessment. Our work is complementary: it does not synthesize new views or classify damage directly. It asks how to compress and reconstruct high-resolution drone imagery so that such visual evidence can move through HPC workflows at lower cost.

B. Learned Compression and Diffusion Reconstruction

Learned image compression commonly optimizes a rate-distortion objective. Hyperprior models improve learned codecs by modeling latent dependencies and side information [3]. CDC differs from deterministic decoder approaches because reconstruction uses a conditional diffusion model [6]. This creates a useful quality path but also a runtime path that depends on denoising steps. Ho et al. [4] established diffusion probabilistic models as iterative denoising generators, and Rombach et al. [5] showed that latent-space diffusion can improve the efficiency of high-resolution synthesis. For our setting, the relevant question is not whether diffusion can reconstruct visually plausible images. It is whether the iterative decoder can be scheduled, tiled, and measured well enough for operational use on large drone-image collections.

C. HPC Workflow Framing

HPC image workflows often fail at practical boundaries: a model may fit on a small test image but become slow or memory-limited at native resolution. DeltaAI is built around NVIDIA GH200 Grace Hopper systems [7], which makes it a relevant platform for evaluating GPU memory pressure and full-resolution reconstruction. We treat platform cost as part of the method. A compression setting is not useful merely because it reports a favorable BPP;

it must also fit GPU memory, reconstruct fast enough, preserve relevant visual structure, and leave auditable evidence for the user.

III. Problem Definition

A. Workflow Objective

Let $x \in [0, 1]^{H \times W \times 3}$ denote a full-resolution drone image. A CDC workflow maps x to a compressed representation z and reconstructs an image $\hat{x}$ through a conditional diffusion decoder. In this paper, the target image size is $H \times W = 3648 \times 5440$ after cropping. The workflow must satisfy three practical requirements:

- 1) Storage reduction: the compressed representation should be much smaller than uncompressed RGB.
- 2) Feasible reconstruction: decoding should fit GPU memory and finish within a usable time budget.
- 3) Inspectability: reconstruction artifacts should be visible to reviewers through tables, heatmaps, and side-by-side comparisons.

B. Metrics

We report compression ratio against uncompressed 8-bit RGB storage:

$$R = \frac{24}{\text{BPP}}, \quad (1)$$

where BPP is the estimated model bitrate. A BPP of 0.35 corresponds to a compression ratio of about $68.6\times$.

For a candidate configuration θ , we also report runtime reduction and memory reduction relative to a baseline:

$$S_T(\theta) = 1 - \frac{T_\theta}{T_{\text{base}}}, \quad (2)$$

$$G_M(\theta) = \frac{M_{\text{base}}}{M_\theta}, \quad (3)$$

where T is wall time per image and M is peak GPU memory. These quantities make the systems trade-off explicit. They do not replace image-quality metrics.

Reconstruction quality is measured with PSNR, SSIM [8], mean absolute error (MAE), and high-percentile absolute error. For tiled runs, we additionally record a seam metric and inspect heatmaps. This combination matters because a configuration can preserve average PSNR and SSIM while producing regular boundary artifacts that are visually important.

IV. Implementation

A. CDC Path

The implementation uses the x-parameterized CDC path adapted in this repository from the original CDC implementation [6]. The x-parameterized path does not expose a direct runtime parameter named `compression_level`, quality, bitrate, or quantization. Compression level is therefore controlled through checkpoint choice. The experiment suite reports checkpoint name together with measured BPP, compression ratio, PSNR, and SSIM so the compression setting is interpretable.

B. Experiment Runner

Each run writes a per-image results.csv, an aggregate summary.csv, a manifest, a short report, and optional visual files. The runner records wall time, GPU-timed inference time, peak GPU memory, BPP, compression ratio, PSNR, SSIM, MSE, RMSE, MAE, 95th and 99th percentile absolute error, maximum absolute error, bias, error standard deviation, and tiled-only seam error. The lightweight GitHub package stores tables and downsampled visuals. Full-resolution reconstructions, checkpoints, raw images, and full logs remain on DeltaAI.

C. Tiling

Native full-resolution decoding is memory-heavy because the full image tensor and diffusion intermediates must be represented together. Tiling splits the image into fixed-size patches, compresses and reconstructs each patch, and stitches the outputs back to the native image size. We evaluate 512×512 , 1024×1024 , and 2048×2048 in the first pilot, then add 256×256 for the selected run. Tiling changes the memory footprint, but it can introduce boundary artifacts. The workflow therefore saves three families of visual evidence:

- original/reconstruction/error comparison panels,
- absolute-error heatmaps,
- visual QA panels with histograms and difference distributions.

V. Experimental Setup

A. Data

The experiments use full-resolution drone images cropped to 5440×3648 pixels. The reconstruction step sweep uses a repeated profiling set. The tiling pilot uses 8 images. The selected tiling validation uses 50 images. The compression-side validation uses 20 images. These sample sizes are sufficient to identify runtime-memory regimes, but they are not yet a final large-scale benchmark. We therefore treat the results as an operating-point study.

B. Platform

The main experiments run on NCSA DeltaAI with GH200-class hardware. The committed results should be read as DeltaAI GH200 results unless a row explicitly states otherwise. A small Delta H200 check is included only as a platform-sensitivity note. It is not treated as a full cross-system benchmark.

C. Experiment Matrix

The experiment suite covers seven questions:

- 1) How does denoising step count affect reconstruction time and quality?
- 2) Does fp16 reduce runtime and memory, and can it be used for BPP reporting?
- 3) How much do resolution reduction and native full-resolution decoding differ?

TABLE I

DeltaAI GH200 reconstruction step sweep, fp32, batch size 1.

Steps	Sec./image	GB	PSNR	SSIM
5	11.27	52.2	31.57	0.9066
10	22.29	52.2	31.00	0.9019
20	44.37	52.2	30.47	0.8961
30	67.16	52.2	30.25	0.8921
50	110.56	52.2	30.03	0.8873
65	143.67	52.2	29.92	0.8845
100	220.85	52.2	29.77	0.8809

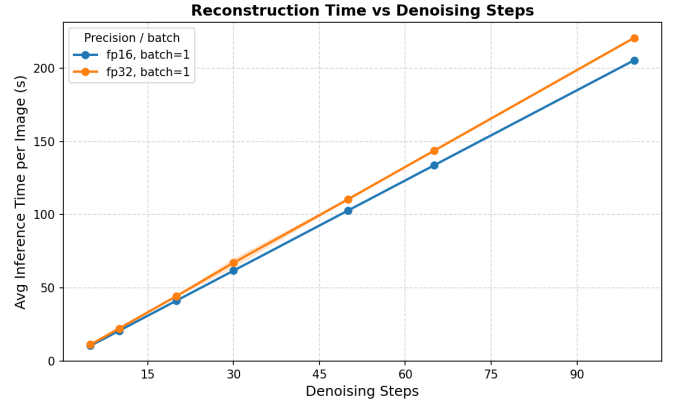


Fig. 1. Measured reconstruction time versus denoising steps. The near-linear trend shows that the iterative decoder dominates runtime.

- 4) Does controlled-resolution batching improve throughput?
- 5) Which checkpoint provides the best compression-quality point?
- 6) Does tiling reduce memory without unacceptable quality loss?
- 7) Does local node storage improve runtime relative to shared storage?

VI. Results

A. Diffusion Steps Dominate Reconstruction Time

Table I reports the DeltaAI GH200 fp32 step sweep. Runtime grows almost linearly with denoising steps. Reducing the denoising process from 65 steps to 5 steps lowers inference time from 143.67 to 11.27 seconds per image, a 92.2% reduction. Peak memory stays near 52.2 GB because the same full-resolution image is decoded. This result separates two problems: denoising steps control time, while resolution and tiling control memory.

B. Precision Helps Memory but Not Current Bitrate Reporting

The fp16 profile lowers peak GPU memory from about 52.2 GB to about 34.2 GB and slightly improves runtime, but the current fp16 output records invalid BPP values. For that reason, fp16 is useful as a speed-memory observation but not as the reporting path for compression-ratio claims in this draft. This is a practical example of why

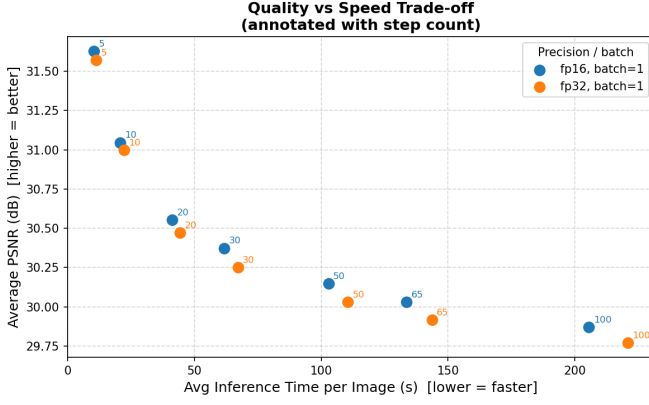


Fig. 2. Quality-speed trade-off for the reconstruction step sweep. Lower step counts improve speed, while PSNR and SSIM change more gradually.

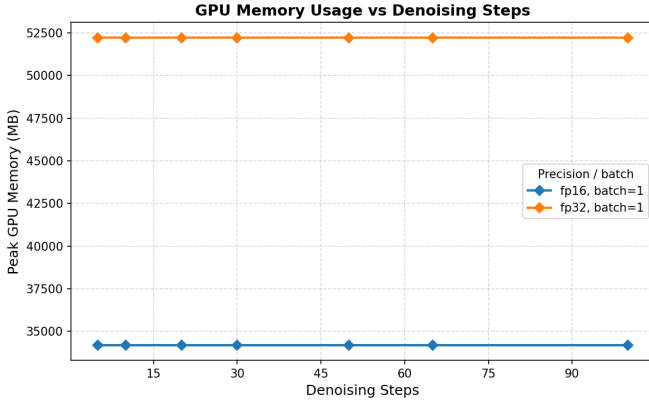


Fig. 3. Peak GPU memory in the reconstruction step sweep. Precision changes memory, but step count alone does not remove the full-image memory footprint.

workflow metrics must include data validity checks, not only runtime.

C. Native Reconstruction Preserves Detail but Is Expensive

The visual examples in Fig. 4 show representative full-image reconstruction outputs at several step counts. The purpose of this figure is not to choose a final quality threshold, but to connect the numerical step sweep to visible behavior. Disaster-response imagery needs this connection because small details may matter to a human reviewer even when aggregate metrics look acceptable.

D. Tiling Reduces Runtime and Memory

The selected tiling validation compares no tiling, 256×256 tiling, and 512×512 tiling on 50 full-resolution images. Table II summarizes the result. The 256×256 setting reduces time from 144.35 to 79.84 seconds per image and peak memory from 52.01 to 1.66 GB. Using Eqs. (2) and (3), this is a 44.7% runtime reduction and a $31.3\times$ peak-memory reduction relative to no tiling.

The 256×256 setting is also faster and smaller than 512×512 tiling. It is about 9.0% faster and uses about 45.0% less memory. The quality penalty relative to 512×512 is modest in scalar metrics: PSNR is lower by 0.086 dB, SSIM is lower by 0.0031, MAE is higher by 0.000217, and error p99 is higher by 0.001314. The seam metric is nearly unchanged. These numbers do not prove that every 256×256 reconstruction is artifact-free, but they justify treating it as the current speed-memory candidate.

E. Visual Checks Are Needed for Tiling

Figure 6 compares the no-tiling, 256×256 , and 512×512 visual panels for the same representative image. These figures are downsampled from experiment outputs, but they serve an important role: they help identify whether tiling creates regular spatial patterns that may not be obvious in the aggregate table. For this sample, the heatmaps concentrate error around high-frequency visual structures rather than showing an obvious global grid pattern.

F. Resolution Reduction Is Fast but Quality-Limited

Resolution reduction gives the largest raw speedup, but it changes the information available to downstream tasks. In the compression-side validation, reducing the longest edge to 4K lowers time from 144.32 to 28.89 seconds per image and peak memory from 52.00 to 28.98 GB. Reducing to 2K lowers time to 7.36 seconds and memory to 7.47 GB, but SSIM drops to 0.689. The 1K setting is faster again, but the quality loss is too large for detailed visual inspection unless a downstream task confirms that such coarse imagery is adequate.

G. Batch Size Does Not Improve the Controlled 2K Run

The controlled 2K batch-size sweep shows that larger batches do not improve throughput in the current implementation. Batch size 1 takes 7.35 seconds per image and uses 7.47 GB. Batch size 2 slows to 11.60 seconds per image and uses 14.64 GB. Batch size 4 takes 9.29 seconds per image and uses 28.96 GB. The result supports batch size 1 as the default for the current workflow. The likely reason is that batching increases memory pressure without increasing effective throughput enough to offset overhead.

H. Checkpoint Choice Controls the Storage-Quality Trade-Off

The x-parameterized CDC path exposes compression level through checkpoint choice. Table V shows representative checkpoint results on native full-resolution images. Runtime stays near 144 seconds per image for all listed checkpoints, so checkpoint choice mainly changes BPP-derived compression ratio and quality. The high-compression b00128 checkpoint reaches $165.73\times$ compression with lower SSIM. The b00512 checkpoint has higher SSIM and a lower compression ratio. The b00064 checkpoint is a balanced candidate, with $93.11\times$ compression and SSIM 0.880.



Fig. 4. Representative reconstruction examples from the step-sweep outputs. These are existing experiment outputs from the repository, copied into the paper folder for compilation.

TABLE II
Selected DeltaAI GH200 tiling validation on 50 full-resolution images, 65 denoising steps, fp32.

Setup	Sec./image	GB	Ratio	PSNR	SSIM	MAE	Error p99	Seam
No tiling	144.35	52.01	70.11×	28.96	0.8616	0.027016	0.117676	n/a
256 × 256 tile	79.84	1.66	68.24×	28.78	0.8552	0.027463	0.120211	0.032556
512 × 512 tile	87.71	3.02	66.31×	28.86	0.8583	0.027246	0.118897	0.032224

TABLE III
Resolution sweep on DeltaAI GH200, 20 images, baseline checkpoint.

Resolution	Sec.	GB	Ratio	PSNR	SSIM
Native	144.32	52.00	71.20×	29.45	0.873
4K	28.89	28.98	72.92×	28.26	0.847
2K	7.36	7.47	81.31×	24.05	0.689
1K	1.94	2.01	77.83×	21.95	0.610

TABLE IV
Controlled 2K batch-size sweep, 20 images.

Batch	Sec./image	GB	PSNR	SSIM
1	7.35	7.47	24.04	0.6886
2	11.60	14.64	24.04	0.6887
4	9.29	28.96	24.04	0.6886

I. Storage Placement Has Little Effect in the Current Run

The storage comparison shows almost no difference between shared project filesystem access and node-local

staged input. Shared storage takes 143.86 seconds per image, while local staging takes 143.89 seconds per image. This suggests that the tested native baseline is compute-

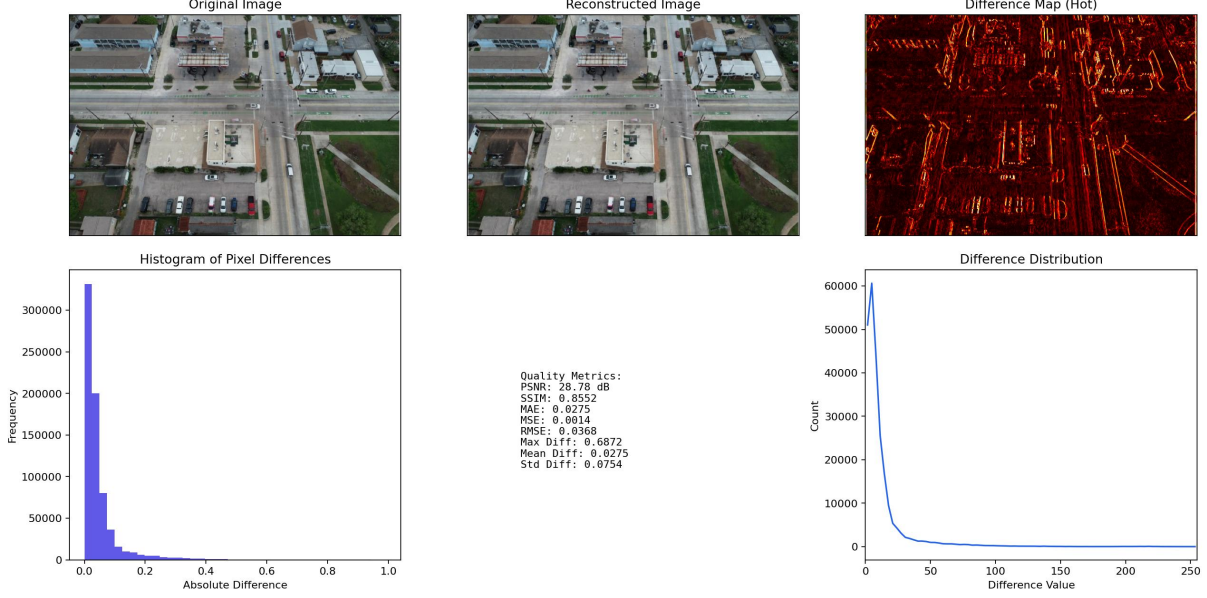


Fig. 5. Visual QA panel for the selected 256×256 tiling case. The panel includes original preview, reconstruction, hot difference map, histogram of pixel differences, quality metrics, and difference distribution.

TABLE V

Checkpoint-controlled compression settings on native full-resolution images, 20-image validation.

Checkpoint	BPP	Sec.	Ratio	PSNR	SSIM
b00512	0.7345	143.73	32.67×	34.84	0.942
b00064	0.2578	143.66	93.11×	33.73	0.880
b00128	0.1448	143.70	165.73×	31.52	0.828
b02048	0.3371	143.70	71.20×	29.45	0.873

TABLE VI

Quick fp32 step comparison. H200 rows are a limited check, not a full benchmark.

System	Steps	Sec./image	PSNR	SSIM
GH200	5	11.27	31.57	0.9066
H200	5	10.87	31.56	0.9066
GH200	20	44.37	30.47	0.8961
H200	20	42.74	30.48	0.8961
GH200	65	143.67	29.92	0.8845
H200	65	138.48	29.92	0.8846

bound rather than I/O-bound. This conclusion should not be generalized to all future runs: larger concurrent jobs or different filesystem pressure could change the result. For the current workflow, however, tiling and denoising-step control are higher-priority optimizations than local staging.

J. Hardware Sensitivity: GH200 and H200 Quick Check

A limited Delta H200 check was run for steps 5, 20, and 65. Table VI compares the matched fp32 rows. H200 is slightly faster in these quick checks, but the difference is small relative to the effect of denoising steps or tiling. We therefore treat hardware comparison as future work rather than a central claim.

VII. Discussion

A. The Main Operating Point

The selected 256×256 tiling configuration is the strongest current operating point because it changes the most limiting systems quantity: memory. Moving from 52.01 GB to 1.66 GB peak memory changes how the

workflow can be scheduled. It lowers the risk of out-of-memory failures, leaves room for larger surrounding pipelines, and makes concurrent processing more realistic. The runtime improvement is also meaningful: a 50-image validation run moves from about two hours at the no-tiling baseline to about 1.1 hours at the selected tiling setting, before considering queue and parallelization effects.

B. Why Not Use Resolution Reduction Alone?

Resolution reduction is tempting because it is fast. The 2K setting is roughly $20\times$ faster than native full-resolution processing in the measured compression-side validation. However, it changes the visual information before compression. For disaster-response inspection, this can be unacceptable: small roof damage, debris lines, street-level obstructions, and building-edge features may disappear. The right question is therefore not "which setting is fastest?" but "which setting preserves the evidence needed by the next task?" Tiling is more attractive because it

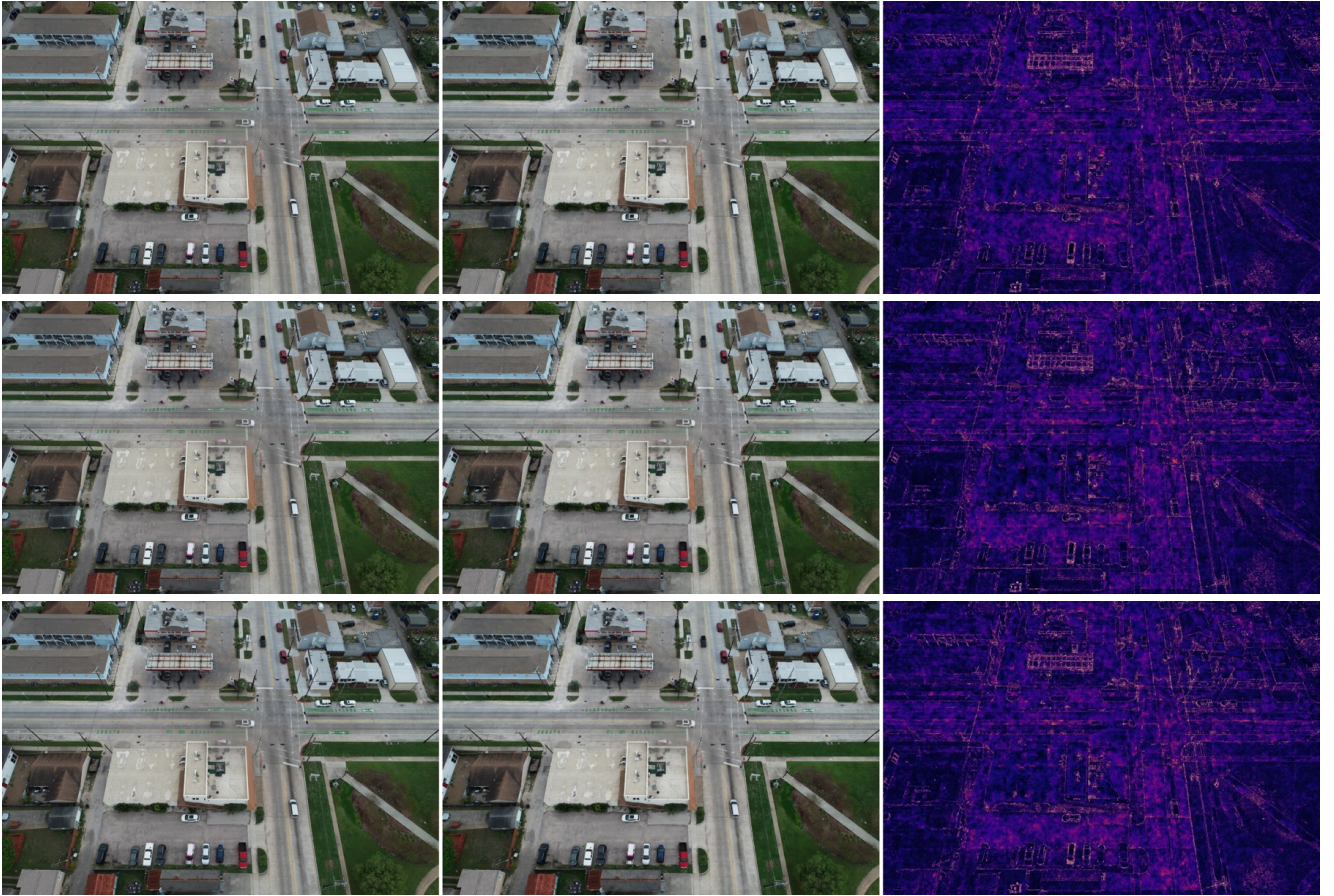


Fig. 6. Existing comparison panels for no tiling, 256×256 tiling, and 512×512 tiling. Each row shows original preview, reconstruction, and absolute-error heatmap for the same representative image.

keeps the native image boundary available while reducing memory pressure.

C. Compression Ratio Is Not Enough

The checkpoint sweep shows why compression ratio cannot be reported alone. The b00128 checkpoint gives a high compression ratio, but at lower SSIM. The b00512 checkpoint gives stronger quality but lower compression. The correct checkpoint depends on whether the downstream user values storage savings, visual fidelity, or classification performance. A future version of this work should choose checkpoints using task-level metrics, not only PSNR and SSIM.

D. Implications for Disaster-Response Workflows

The disaster-imagery setting makes hallucination and loss of detail especially important. Generative visual systems can produce plausible outputs that do not preserve critical structural evidence [1]. Damage-assessment models can also depend on subtle pre- and post-event differences [2]. A CDC workflow for this domain should therefore be conservative: it should expose heatmaps, preserve raw links to original data, and mark configurations that have not been validated with downstream tasks.

VIII. Limitations and Threats to Validity

This draft has four main limitations. First, the selected validation uses 50 full-resolution images and the compression-side validation uses 20 images. These runs identify promising operating points, but they are not yet a statistically broad benchmark across many disaster scenes. Second, the visual QA panels show representative examples. A final study should inspect more images and report artifact frequencies. Third, the current metrics are image-level fidelity metrics, not downstream damage-assessment accuracy. The final recommendation should be tested with tasks such as road obstruction detection, building-damage classification, or human visual inspection. Fourth, the fp16 path is not ready for compression-ratio reporting because BPP values are invalid in the current outputs. This is an implementation issue, not evidence that fp16 is unusable.

IX. Future Work

The next version of this paper should add three experiments. First, run the selected 256×256 and 512×512 settings on a larger sample and include artifact-frequency reporting. Second, evaluate downstream task performance using disaster-analysis tasks related to street-view and

cross-view damage assessment. Third, separate model compression choices from systems choices by testing checkpoint selection under the same tiling and denoising-step settings.

X. Conclusion

This paper evaluates CDC as a cost-aware HPC workflow for high-resolution drone imagery. The central finding is practical: native full-image CDC is feasible on DeltaAI GH200, but it is too slow and memory-heavy to be the default operating point. Tiling changes the cost profile. On 50 full-resolution images, 256×256 tiling reduces runtime by 44.7% and peak memory by $31.3\times$ relative to no tiling, with small scalar quality losses relative to 512×512 tiling. Resolution reduction and checkpoint selection remain useful controls, but they should be chosen with downstream visual-evidence needs in mind. The broader lesson is that neural compression for disaster imagery should be evaluated as an end-to-end system: storage, runtime, memory, fidelity, and visual artifact evidence all belong in the same table.

Acknowledgment

The authors thank the NCSA DeltaAI support ecosystem and project collaborators for discussions that shaped the experiment design.

References

- [1] Y. Yang, L. Zou, and W. Jepson, “Satellite-to-street: Synthesizing post-disaster views from satellite imagery via generative vision models,” arXiv preprint arXiv:2603.20697, 2026. [Online]. Available: <https://arxiv.org/abs/2603.20697>
- [2] Y. Yang, L. Zou, B. Zhou, D. Li, B. Lin, J. Abedin, and M. Yang, “Hyperlocal disaster damage assessment using bi-temporal street-view imagery and pre-trained vision models,” *Computers, Environment and Urban Systems*, vol. 121, p. 102335, 2025.
- [3] J. Ballé, D. Minnen, S. Singh, S. J. Hwang, and N. Johnston, “Variational image compression with a scale hyperprior,” in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=rkcQFMZRb>
- [4] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 6840–6851. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>
- [5] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10 684–10 695. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2022/html/Rombach_High-Resolution_Image_Synthesis_With_Latent_Diffusion_Models_CVPR_2022_paper.html
- [6] R. Yang and S. Mandt, “Lossy image compression with conditional diffusion models,” arXiv preprint arXiv:2209.06950, 2022. [Online]. Available: <https://arxiv.org/abs/2209.06950>
- [7] National Center for Supercomputing Applications, “DeltaAI User Documentation,” <https://docs.ncsa.illinois.edu/systems/deltaai/>, 2026, accessed 2026-05-30.
- [8] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.